

WORKSHEET MAHASISWA

Modul 2 — Vanilla JavaScript dan Interaktivitas Web

Dasar Java Script · DOM dan Event · Validasi Form · Asynchronous · Debugging

Mata Kuliah Proyek 3 — Proyek Pengembangan Perangkat Lunak Berbasis Web · D3 Teknik Informatika

Identitas	Isian
Nama	Hasya Nadana Hakim
NIM	251511040
Kelas	D3 2B
Tanggal Mulai	14 September 2026
Tanggal Pengumpulan	18 September 2026
URL Repository	https://github.com/shiningsstars/Proyek-3-Module-2.git

Petunjuk Penggunaan

- ☐ Isi worksheet bersamaan dengan pengerjaan modul, bukan setelah seluruh tugas selesai.
- ☐ Pada eksperimen, tulis prediksi sebelum menjalankan kode, lalu catat hasil aktual dan bukti.
- ☐ Gunakan Console, Elements, Network, dan tampilan halaman sebagai sumber bukti sesuai aktivitas.
- ☐ Gunakan hint secara bertahap; catat percobaan sebelum membuka hint berikutnya.
- ☐ Catat penggunaan AI atau sumber bantuan secara transparan dan lakukan verifikasi mandiri.
- ☐ Worksheet dikumpulkan dalam PDF bersama bukti yang disyaratkan pada tugas.

Alur Belajar Alur pembelajaran dilaksanakan melalui lima tahap, yaitu membuat prediksi, menjalankan percobaan, mengamati hasil, menjelaskan temuan, dan melakukan perbaikan. Bukti proses memiliki kedudukan yang sama pentingnya dengan hasil akhir.

A. Eksperimen

A1. Eksperimen Console dan Tipe Data

Isi kolom *Prediksi* sebelum membuka *Console*. Setelah menjalankan kode, catat hasil aktual dan alasan perbedaannya.

Ekspresi atau Pemeriksaan	Prediksi	Hasil Aktual	Tipe Data atau Makna	Penjelasan
typeof nama	string	string	Tipe data berupa teks	nama berisi "Rani" sehingga tipe datanya string
typeof nilaiTeks	string	string	Tipe data berupa teks	nilaiTeks berisi "80" sehingga tipe datanya string
typeof nilaiAngka	number	number	Tipe data berupa angka	nilaiAngka berisi 80 sehingga tipe datanya number
nilaiTeks + 5	"805"	"805"	Tipe data gabungan teks dan angka	nilaiTeks merupakan string sehingga operator + menggabungkan "80" dengan 5
Number(nilaiTeks) + 5	85	85	Penjumlahan angka setelah teks dikonversi menjadi angka	Number mengubah teks '80' menjadi angka 80, lalu dijumlahkan dengan angka 5
nilaiTeks == nilaiAngka	true	true	Nilai dianggap sama	== melakukan konversi tipe data sehingga "80" dianggap sama dengan 80
nilaiTeks === nilaiAngka	false	false	Nilai dan tipe data berbeda	=== membandingkan nilai dan tipe data, sehingga "80" tidak sama dengan 80
Number(teks bukan angka)		NaN	Teks tidak dapat dikonversi menjadi angka	Number() menghasilkan NaN ketika teks tidak bisa diubah menjadi angka

Apa bukti bahwa angka 80 dan teks "80" berbeda?

typeof "80" menghasilkan string, sedangkan typeof 80 menghasilkan number. "80" === 80 menghasilkan false karena tipe data yang berbeda.

Mengapa operator + dapat menghasilkan hasil yang berbeda?

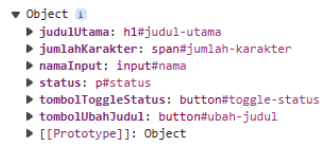
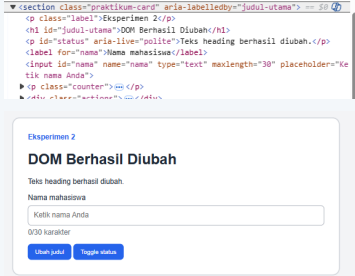
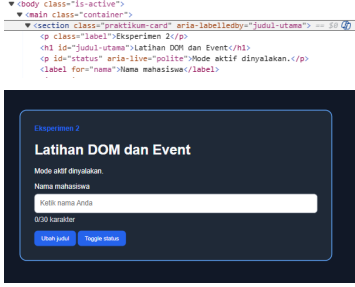
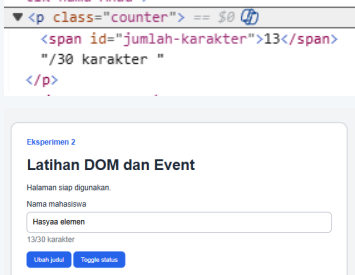
Operasi + dapat digunakan untuk penjumlahan angka atau string. Jika salah satu bertipe string seperti "80" + 5, hasilnya menjadi "805" karena teks "80" ditambahkan dengan angka 5.

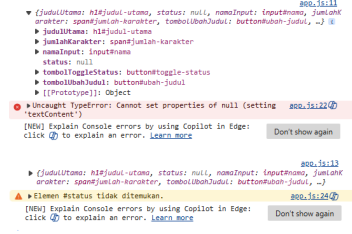
Mengapa perbandingan ketat === menjadi pilihan awal yang lebih aman?

Karena === membandingkan nilai dan tipe data tanpa melakukan konversi secara otomatis. Sehingga hasil perbandingan lebih jelas dan mengurangi kemungkinan kesalahan.

A2. Eksperimen DOM dan Event

Alur yang diamati Alur yang diamati dimulai ketika pengguna melakukan suatu aksi. Browser kemudian membangkitkan event, fungsi penanganan dijalankan, elemen DOM atau class diperbarui, dan browser merender perubahan pada tampilan.

Langkah	Prediksi sebelum diuji	Hasil aktual	Bukti dan Penjelasan
1 · Memilih elemen	Keenam selector akan menemukan elemen yang sesuai sehingga tidak menghasilkan null	Keenam elemen berhasil ditemukan dan ditampilkan di console	 <pre> Object judulUtama: h1#judul-utama jumlahKarakter: span#jumlah-karakter namaInput: input#nama status: p#status tombolToggleStatus: button#toggle-status tombolUbahJudul: button#ubah-judul [[Prototype]]: Object </pre>
2 · Mengubah heading	Setelah tombol 'Ubah Judul' diklik, teks heading akan berubah menjadi 'DOM Berhasil Diubah' dan status ikut berubah	Heading berubah menjadi 'DOM Berhasil Diubah' dan status berubah	 <pre> <section class="praktikum-card" aria-labelledby="judul-utama"> <p class="label">Eksperimen 2</p> <h1 id="judul-utama">DOM Berhasil Diubah</h1> <p id="status" aria-live="polite">Teks heading berhasil diubah.</p> <label for="nama">Nama mahasiswa</label> <input id="nama" name="nama" type="text" maxlength="30" placeholder="Ketik nama Anda"> <p class="counter"></p> <button class="tombol-ubah-judul">Ubah Judul</button> <button class="tombol-toggl-status">Toggle status</button> </pre>
3 · Mengubah class dan atribut aria-pressed	Setiap tombol Toggle status diklik, class is-active akan ditambahkan dan aria-pressed akan berubah	Class is-active, nilai aria-pressed dan tampilan halaman berubah setiap tombol diklik	 <pre> <body class="is-active"> <main class="container"> <section class="praktikum-card" aria-labelledby="judul-utama"> <p class="label">Eksperimen 2</p> <h1 id="judul-utama">Latihan DOM dan Event</h1> <p id="status" aria-live="polite">Mode aktif dinyalakan.</p> <label for="nama">Nama mahasiswa</label> </pre>
4 · Event input dan penghitung karakter	Penghitungan karakter berubah setiap diisi input	Jumlah karakter bertambah saat mengetik, berkurang saat menghapus dan berubah sesuai dengan teks yang ditempelkan	 <pre> <p class="counter"> 13 "/30 karakter " </p> </pre>

Langkah	Prediksi sebelum diuji	Hasil aktual	Bukti dan Penjelasan
5 · Selector tidak cocok dan menghasilkan nilai null	Jika id="status" diubah menjadi id="status-pesan", selector #status menghasilkan null dan penggunaan status.textContent akan menyebabkan error	Sebelum pemeriksaan null, Console menunjukkan error karena status bernilai null. Setelah menggunakan ubahStatus(), Console menunjukkan peringatan Elemen #status tidak ditemukan dan program tidak mengalami error yang sama	

Tuliskan pesan error ketika selector tidak cocok, meliputi nama file, nomor baris, dan inti pesan:

```

app.js:11
{judulUtama: h1#judul-utama, status: null, namaInput: input#nama, jumlahK
arakter: span#jumlah-karakter, tombolUbahJudul: button#ubah-judul, ...}
  judulUtama: h1#judul-utama
  jumlahKarakter: span#jumlah-karakter
  namaInput: input#nama
  status: null
  tombolToggleStatus: button#toggle-status
  tombolUbahJudul: button#ubah-judul
  [[Prototype]]: Object
Uncaught TypeError: Cannot set properties of null (setting 'textContent')
app.js:22:4
[NEW] Explain Console errors by using Copilot in Edge:
click to explain an error. Learn more
Don't show again

```

File: app.js

Nomor baris: 22

Inti pesan: Cannot set properties of null (setting 'textContent')

Jelaskan perbedaan bukti yang diperoleh sebelum dan sesudah pemeriksaan nilai null melalui function ubahStatus.

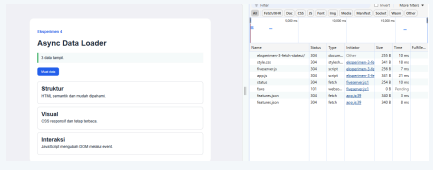
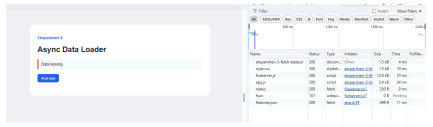
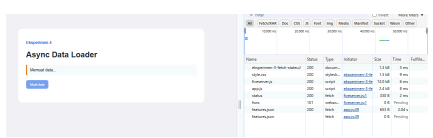
Sebelum pemeriksaan null, status bernilai null ketika selector #status tidak menemukan elemen. Saat status.textContent dijalankan, terjadi error karena program mencoba mengakses textContent dari nilai null. Setelah menggunakan function ubahStatus(), program memeriksa terlebih dahulu apakah status bernilai null. Jika elemen tidak ditemukan, program menampilkan pesan peringatan dan tidak menjalankan perubahan textContent, sehingga error dapat dicegah

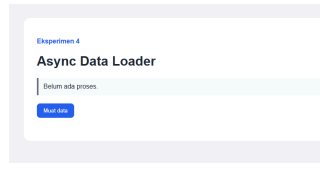
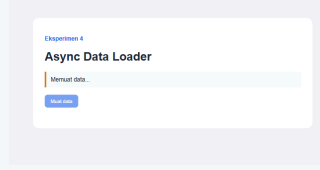
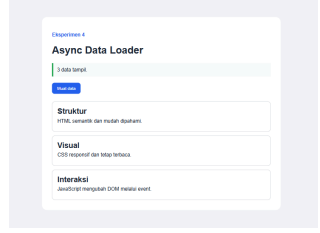
Jelaskan pembagian peran JavaScript dan CSS ketika class is-active diubah melalui mekanisme toggle.

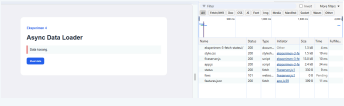
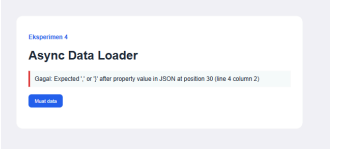
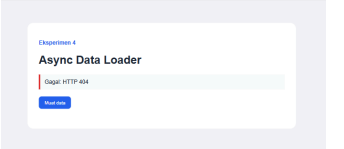
JavaScript mengatur status, sedangkan CSS mengatur tampilan. JavaScript bertugas mengatur perilaku halaman dengan menambahkan atau menghapus class is-active menggunakan classList.toggle(). CSS bertugas mengatur tampilan berdasarkan keberadaan class tersebut. Ketika is-active ditambahkan pada <body>, aturan body.is-active diterapkan sehingga tampilan berubah

A3. Eksperimen Promise, fetch, dan State UI

Jalankan proyek melalui server pengembangan lokal. Gunakan tampilan halaman, Console, dan Network sebagai bukti pengujian.

Skenario atau Langkah	Prediksi	Hasil Aktual	Bukti dari Console, Network, atau Antarmuka
Urutan A, B, C pada setTimeout(0)	Output berurutan dari A, B, C	Berurutan A, B, C	<pre> A: mulai app.js:26 B: akhir app.js:30 C: timer selesai app.js:28 </pre>
Data JSON valid	Data berhasil diambil	Success dan tiga kartu tampil	
Array kosong []	Data kosong	Array kosong	
Path salah atau JSON rusak	Path salah menghasilkan error HTTP dan JSON rusak menghasilkan error	Keduanya masuk state error	
Klik berulang saat loading	Tombol menjadi tidak aktif selama proses loading	Tombol tidak dapat diklik kembali sampai proses loading selesai	

State	Apa yang tampil	Kapan terjadi	Bukti
idle	Tampilan awal dan tombol 'Muat Data' aktif	Sebelum proses pemuatan dimulai	
loading	Halaman menampilkan pesan 'Memuat data...'	Saat data sedang melakukan fetch pada jaringan yang tidak stabil	
success	Pesan '3 data tampil' dan tiga kartu fitur	Saat data berhasil diambil	

State	Apa yang tampil	Kapan terjadi	Bukti
empty	Pesan 'Data kosong'	Saat data berhasil diambil tetapi array kosong	
error	Pesan gagal	Saat terjadi error pada path atau parsing JSON	  <pre> app.js:19 GET https://127.0.0.1:5580/eksperimen-3-fetch-status/data/features.json 404 (Not Found) app.js:63 Error: HTTP 404 at XMLHttpRequest.onerror (app.js:63:15) at XMLHttpRequest.onerror (app.js:52:15) </pre>

Mengapa delay 0 pada setTimeout tetap tidak membuat callback berjalan sebelum kode sinkron selesai?

Karena JavaScript baru mengeksekusi callback setelah kode sinkron yang sedang berjalan selesai sepenuhnya.

Apa perbedaan antara error HTTP atau kesalahan path dengan error parsing JSON berdasarkan bukti yang Anda peroleh?

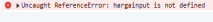
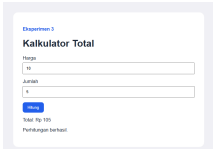
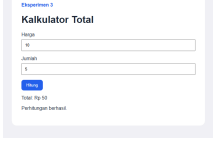
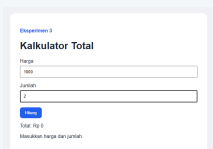
Error HTTP atau kesalahan path terjadi ketika alamat file tidak ditemukan sedangkan error parsing JSON terjadi ketika file berhasil diambil, tetapi isi JSON tidak valid sehingga response.json() gagal.

Mengapa finally cocok untuk mengaktifkan kembali tombol?

Karena finally akan selalu mengeksekusi baik ketika proses try berhasil ataupun tidak.

A4. Eksperimen Diagnosis Error JavaScript

Metode Diagnosis Metode diagnosis dimulai dengan mencatat gejala dan hasil yang diharapkan, mereproduksi masalah, membaca pesan error pertama, memeriksa nilai atau selector, menyusun dugaan penyebab, melakukan satu perubahan, kemudian menguji ulang hasil perbaikan.

Kasus	Gejala dan Hasil yang Diharapkan	Pesan Error atau Nilai Aktual	Dugaan Penyebab	Perbaikan	Bukti Uji Ulang
A · Selector	Form tidak dapat diproses karena elemen tidak ditemukan.	Muncul error karena elemen form tidak ditemukan.	id pada HTML tidak sesuai dengan selector #form-hitung pada app.js	Mengembalikan id="form-hitung"	 prosesForm: 1 console.log(form) kembali menunjukkan elemen form dan baseline berhasil.
B · Nama variabel	Submit form menghasilkan error karena variabel harga tidak dikenali.	ReferenceError: hargainput is not defined.	Kapitalisasi nama variabel salah: hargaInput ditulis hargainput.	Mengembalikan hargainput menjadi hargaInput.	 prosesForm: 1 Submit kembali berhasil dan total dapat dihitung.
C · Tipe data dan operator	Hasil perhitungan menjadi penggabungan teks.	typeof harga dan typeof jumlah = string; harga 10 dan jumlah 5 menghasilkan 105.	Nilai input berupa string dan konversi Number() dihilangkan, kemudian operator + digunakan.	Mengembalikan Number() dan operator *.	  Harga 10 dan jumlah 5 menghasilkan total 50.
D · Form reload	Setelah submit, halaman seharusnya tidak dimuat ulang.	Halaman melakukan reload dan nilai input kembali ke kondisi awal.	event.preventDefault() dikomentari sehingga perilaku default form berjalan.	Mengaktifkan kembali event.preventDefault().	 Setelah diperbaiki, submit tidak memuat ulang halaman dan hasil tetap tampil.

Kasus	Gejala dan Hasil yang Diharapkan	Pesan Error atau Nilai Aktual	Dugaan Penyebab	Perbaikan	Bukti Uji Ulang
E · Handler ganda	Satu kali submit seharusnya menjalankan prosesForm satu kali.	prosesForm: 2 setelah satu kali submit.	Terdapat dua listener submit yang sama-sama memanggil prosesForm().	Menghapus listener tambahan.	prosesForm: 1 prosesForm: 2 Setelah diperbaiki, satu submit menghasilkan prosesForm: 1.

Kasus paling membantu pemahaman saya dan mengapa:

Kasus C (Tipe data dan operator) paling membantu pemahaman saya karena saya dapat melihat bahwa nilai dari input berupa string. Ketika Number() dihilangkan dan operator + digunakan, nilai 10 dan 5 menjadi 105. Dari percobaan ini saya memahami pentingnya konversi tipe data dan penggunaan operator yang sesuai dalam JavaScript.

B. Pertanyaan Pemahaman

B1. Logika, Function, Array, dan Object

Kapan let lebih tepat digunakan daripada const?

let digunakan ketika nilai sebuah variabel masih perlu diubah selama program berjalan. Sedangkan const digunakan jika nilai variabel tidak akan diubah.

Mengapa nilai dari elemen input perlu dikonversi sebelum perhitungan?

Karena nilai dari input biasanya terbaca sebagai string. Jadi perlu diubah ke number menggunakan Number() supaya bisa dihitung dengan benar.

Mengapa kondisi kategori atau diskon dengan batas terbesar diperiksa lebih dahulu?

Supaya nilai yang masuk kategori paling tinggi tidak terlanjur masuk ke kategori yang lebih rendah. Jadi pengecekannya dimulai dari batas terbesar ke yang lebih kecil.

Apa perbedaan parameter, argument, dan return?

Parameter adalah variabel yang ada di dalam function. Argument adalah nilai yang diberikan saat function dipanggil. Return adalah hasil yang dikembalikan oleh function.

Mengapa function logika sebaiknya dipisahkan dari function yang mengubah tampilan?

Supaya kode lebih rapi dan mudah diperbaiki. Function logika fokus mengolah data, sedangkan function tampilan fokus menampilkan hasilnya.

Mengapa variabel lokal tidak dapat diakses dari luar function?

Karena variabel lokal hanya berlaku di dalam function tempat variabel tersebut dibuat. Di luar function, variabel itu tidak dikenali.

Mengapa kumpulan entitas dengan struktur sama cocok disimpan sebagai array of objects?

Karena setiap object bisa menyimpan data satu entitas, sedangkan array bisa menampung banyak object dengan struktur yang sama. Jadi datanya lebih mudah dikelola

Apa akibat mengakses indeks array yang tidak tersedia?

Hasilnya undefined karena tidak ada data pada indeks tersebut.

B2. DOM, Event, dan Validasi Form

Apa yang dikembalikan querySelector ketika selector tidak menemukan elemen?

null

Mengapa classList lebih tepat untuk perubahan visual daripada menulis style satu per satu di JavaScript?

Karena classList bisa menambah atau menghapus class, lalu perubahan tampilannya diatur melalui CSS. Jadi JavaScript tidak perlu mengatur banyak style secara langsung.

Apa hubungan antara addEventListener, event handler, dan perubahan DOM?

addEventListener digunakan untuk menunggu suatu event, lalu event handler menjalankan kode ketika event terjadi. Kode tersebut bisa digunakan untuk mengubah DOM, misalnya mengubah teks atau class.

Mengapa atribut defer digunakan ketika app.js dipanggil dari head?

Agar file JavaScript tetap bisa dimuat tanpa menghentikan proses membaca HTML. Script baru dijalankan setelah HTML selesai diproses.

Mengapa textContent digunakan untuk teks biasa atau data input pengguna?

Karena textContent memasukkan data sebagai teks biasa, sehingga isi dari input tidak dianggap sebagai kode HTML.

Apa fungsi event.preventDefault() pada submit form?

Untuk mencegah perilaku bawaan form, yaitu memuat ulang atau mengirim halaman. Dengan begitu, proses submit bisa ditangani sendiri menggunakan JavaScript.

Mengapa trim digunakan sebelum memvalidasi input teks?

Untuk menghapus spasi di awal dan akhir teks. Jadi input yang hanya berisi spasi tidak dianggap sebagai teks yang valid.

Mengapa validasi sisi klien tetap tidak menggantikan validasi server?

Karena validasi di browser masih bisa dilewati atau dimanipulasi oleh pengguna. Validasi server tetap diperlukan untuk memastikan data yang diterima benar dan aman.

B3. Asynchronous JavaScript

Apa perbedaan state Promise pending, fulfilled, dan rejected?

pending berarti proses masih berjalan. fulfilled berarti proses berhasil dan menghasilkan nilai. rejected berarti proses gagal dan menghasilkan error.

Bagian mana yang ditunda oleh await dan bagian mana yang tetap dapat merespons pengguna?

await menunda proses di dalam function async sampai Promise selesai. Tetapi browser tetap bisa menjalankan proses lain, seperti menerima klik atau interaksi pengguna.

Mengapa fetch dengan status HTTP 404 tidak otomatis masuk ke catch?

Karena fetch menganggap respons HTTP 404 sebagai respons yang berhasil diterima, bukan error jaringan. Jadi perlu mengecek response.ok dan membuat error sendiri jika nilainya false.

Apa fungsi `response.ok` sebelum `response.json()` dipanggil?

Untuk mengecek apakah respons HTTP berhasil. Kalau `response.ok` bernilai `false`, program bisa menghentikan proses dan menampilkan error sebelum mencoba membaca JSON.

Apa perbedaan `state empty` dan `error` dari sudut pandang pengguna?

`empty` berarti data berhasil diambil tetapi memang tidak ada data. Sedangkan `error` berarti data gagal diambil atau diproses.

Mengapa tombol dinonaktifkan selama proses permintaan data berlangsung?

Supaya pengguna tidak menekan tombol berkali-kali dan membuat beberapa permintaan data berjalan bersamaan

Mengapa `state loading` harus selalu diakhiri setelah proses selesai, baik berhasil maupun gagal?

Supaya tampilan tidak berhenti di kondisi `loading`. Setelah proses selesai, halaman harus menampilkan hasil `success`, `empty`, atau `error`, dan tombol bisa digunakan kembali.

Bagaimana `Console` dan `Network` saling melengkapi ketika mendiagnosis `fetch`?

`Network` digunakan untuk melihat permintaan seperti URL dan status HTTP, sedangkan `Console` digunakan untuk melihat hasil, nilai data, dan pesan error. Keduanya membantu mengetahui apakah masalahnya ada pada permintaan atau saat data diproses.

B4. Debugging

Mengapa error pertama pada `Console` diperiksa sebelum error berikutnya?

Karena error pertama biasanya menjadi penyebab munculnya error berikutnya. Dengan memperbaiki error pertama dulu, kita bisa mengetahui masalah sebenarnya tanpa tertutup oleh error lain.

Apa perbedaan gejala, penyebab, dugaan, dan bukti?

Gejala adalah masalah yang terlihat. Penyebab adalah hal yang membuat masalah terjadi. Dugaan adalah perkiraan penyebab sebelum dibuktikan. Bukti adalah hasil pengujian yang mendukung atau membuktikan dugaan tersebut.

Bagaimana `typeof` membantu diagnosis kesalahan perhitungan?

`typeof` digunakan untuk melihat tipe data suatu nilai. Misalnya, jika hasilnya `string`, berarti nilai input masih berupa teks dan perlu dikonversi ke `number` sebelum dihitung.

Bagaimana `console.count` membuktikan handler berjalan lebih dari sekali?

`console.count` menghitung berapa kali bagian kode dijalankan. Jika satu kali `submit` menghasilkan hitungan 2, berarti handler atau function tersebut dijalankan dua kali.

Mengapa hanya satu perubahan dilakukan pada setiap percobaan perbaikan?

Supaya kita tahu perubahan mana yang benar-benar memperbaiki masalah. Kalau banyak bagian diubah sekaligus, akan sulit mengetahui penyebab dan perbaikannya.

Apa bukti minimum bahwa perbaikan benar-benar berhasil dan bukan hanya menyembunyikan gejala?

Program harus kembali berjalan sesuai hasil yang diharapkan dan kasus yang sebelumnya bermasalah harus bisa diuji ulang tanpa error.

C. Independent Challenge

C1. Kalkulator Harga

Gunakan program pada Console dan function yang memiliki nilai kembalian. Lakukan sedikitnya lima kasus pengujian, termasuk nilai batas Rp100.000, Rp200.000, serta satu input yang tidak valid.

Keputusan	Isian Mahasiswa
Data input dan tipe yang digunakan	Harga dan jumlah menggunakan number, sedangkan status anggota menggunakan boolean.
Function yang dibuat dan tanggung jawabnya	hitungSubtotal() menghitung subtotal, hitungDiskon() menentukan persentase diskon, hitungTotal() menghitung total pembayaran, dan hitungHarga() menggabungkan seluruh perhitungan.
Aturan diskon 10%, 20%, tambahan anggota 5%	Subtotal minimal Rp100.000 mendapat diskon 10%, minimal Rp200.000 mendapat 20%, dan anggota mendapat tambahan 5%.
Cara membatasi total diskon maksimal 25%	Setelah diskon anggota ditambahkan, jika total diskon lebih dari 25%, nilainya diubah menjadi 25%.
Cara menangani harga atau jumlah yang kurang dari atau sama dengan 0	Data dianggap tidak valid dan function mengembalikan pesan bahwa harga dan jumlah harus lebih dari 0.

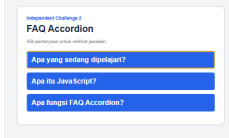
Kasus Uji	Harga	Jumlah	Status Anggota	Ekspektasi	Aktual	Status
1	Rp50.000	1	Bukan anggota	Subtotal Rp50.000, diskon 0%, total Rp50.000	Subtotal Rp50.000, diskon 0%, total Rp50.000	☐ Lulus ☐ Gagal
2 · batas Rp100.000	Rp100.000	1	Bukan anggota	Subtotal Rp100.000, diskon 10%, total Rp90.000	Subtotal Rp100.000, diskon 10%, total Rp90.000	☐ Lulus ☐ Gagal
3 · batas Rp200.000	Rp200.000	1	Bukan anggota	Subtotal Rp200.000, diskon 20%, total Rp160.000	Subtotal Rp200.000, diskon 20%, total Rp160.000	☐ Lulus ☐ Gagal
4	Rp100.000	2	Anggota	Subtotal Rp200.000, diskon 25%, total Rp150.000	Subtotal Rp200.000, diskon 25%, total Rp150.000	☐ Lulus ☐ Gagal
5 · tidak valid	Rp0	2	Bukan anggota	Data ditolak karena harga tidak lebih dari 0	Data ditolak dan muncul pesan error	☐ Lulus ☐ Gagal

Jelaskan kasus nilai batas yang paling penting beserta alasannya:

Kasus batas yang paling penting adalah Rp100.000 dan Rp200.000 karena kedua nilai tersebut menentukan perubahan persentase diskon. Pada Rp100.000 diskon berubah menjadi 10%, sedangkan pada Rp200.000 diskon berubah menjadi 20%. Pengujian ini memastikan kondisi if dan else if berjalan sesuai aturan yang diberikan.

C2. FAQ Accordion

- ☐ Tiga item FAQ dibuat dengan struktur semantik.
- ☐ Pertanyaan menggunakan button, bukan div yang diberi click handler.
- ☐ Event dipasang dengan addEventListener dan tampilan berubah melalui classList.
- ☐ aria-expanded berubah sesuai keadaan.
- ☐ Hanya satu jawaban terbuka pada satu waktu.
- ☐ Dapat digunakan melalui keyboard tanpa library accordion.

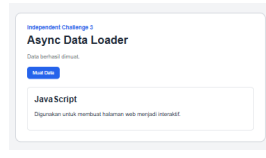
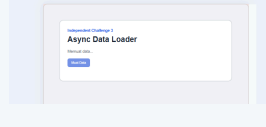
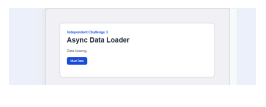
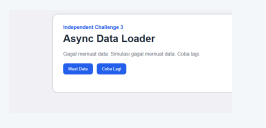
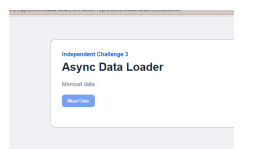
Skenario Uji	Ekspektasi	Hasil Aktual	Bukti
Halaman awal	Semua jawaban tertutup	Semua jawaban FAQ tertutup	
Klik FAQ pertama	Jawaban 1 terbuka; aria-expanded=true	Jawaban pertama terbuka dan aria-expanded berubah menjadi true	
Klik FAQ kedua	Jawaban 2 terbuka; FAQ 1 tertutup	Jawaban kedua terbuka dan jawaban pertama tertutup	
Klik FAQ aktif lagi	State berubah secara konsisten	Jawaban yang terbuka tertutup kembali	
Keyboard	Navigasi menggunakan Tab serta aktivasi menggunakan Enter atau Space berfungsi	Semua pertanyaan FAQ dapat dibuka dan ditutup menggunakan keyboard	

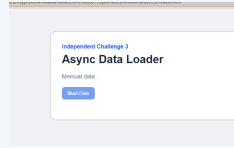
Bagaimana Anda memastikan hanya satu jawaban terbuka pada satu waktu?

Saya menutup semua jawaban terlebih dahulu dengan `classList.remove('is-open')` dan mengubah `aria-expanded` menjadi `false`. Setelah itu, hanya jawaban yang dipilih yang dibuka dengan `classList.add('is-open')` dan `aria-expanded` menjadi `true`.

C3. Async Data Loader

Aspek Desain	Keputusan Mahasiswa
Sumber data JSON lokal	Menggunakan file data/data.json sebagai sumber data.
Delay buatan selama 500 sampai 1500 ms	Menggunakan setTimeout() melalui fungsi delay() dengan waktu acak 500–1500 ms.
Cara memilih satu item secara acak	Menggunakan Math.random() untuk menentukan index data yang akan ditampilkan.
Simulasi kegagalan sebesar 30%	Menggunakan Math.random() < 0.3 untuk menghasilkan simulasi error sebesar 30%.
State loading, empty, error, dan mekanisme percobaan ulang	Menggunakan state loading, empty, error, dan success. Saat terjadi error, pesan Coba lagi ditampilkan dan tombol Coba Lagi dapat menjalankan proses pemuatan kembali.
Cara mencegah klik berulang agar hasil render tetap konsisten	Tombol dinonaktifkan selama proses pemuatan berlangsung dan daftar dikosongkan sebelum render menggunakan replaceChildren().

Skenario Uji	Ekspektasi	Hasil Aktual	Bukti
Pemuatan normal	Data berhasil dimuat dan satu item ditampilkan.	Satu item dari JSON berhasil ditampilkan.	
Loading terlihat	Pesan 'Memuat data...' muncul selama proses delay.	State loading tampil sebelum data selesai dimuat.	
Data kosong	Jika JSON berisi array kosong, state berubah menjadi empty dan tidak ada data yang ditampilkan.	Pesan 'Data kosong.' tampil dan tidak ada item.	
Simulasi gagal	Jika kondisi error 30% terpenuhi, state berubah menjadi error.	Pesan gagal memuat data dan Coba lagi. ditampilkan.	
Tombol Coba Lagi	Tombol menjalankan proses pemuatan data kembali.	Data dimuat kembali setelah tombol Coba Lagi diklik.	

Skenario Uji	Ekspektasi	Hasil Aktual	Bukti
Klik berulang	Klik berulang tidak menyebabkan proses atau hasil render menjadi ganda.	Tombol dinonaktifkan selama proses loading dan data tidak terduplikasi.	

Jelaskan alur Promise dan proses asinkron sejak tombol diklik hingga data berhasil ditampilkan atau error ditangani.

Saat tombol Muat Data diklik, fungsi `muatData()` dijalankan dan state berubah menjadi loading. Setelah itu, fungsi `ambilData()` dipanggil. Fungsi tersebut menggunakan Promise melalui `delay()` untuk memberikan jeda 500–1500 ms sebelum data diproses.

Setelah Promise selesai, data JSON diambil menggunakan `fetch()` dan `await`. Jika data berhasil diperoleh, satu item dipilih secara acak lalu ditampilkan, kemudian state berubah menjadi success. Jika data kosong, state berubah menjadi empty.

Jika terjadi kegagalan, Promise masuk ke bagian catch, kemudian state berubah menjadi error dan pesan Coba lagi ditampilkan. Tombol Coba Lagi dapat menjalankan `muatData()` kembali. Selama proses berlangsung, tombol dinonaktifkan agar klik berulang tidak menyebabkan proses render menjadi ganda.

D. Tugas Praktikum

D1. Task 1 — JavaScript Logic Builder

Target: menggunakan function murni tanpa DOM untuk melakukan validasi nilai, menentukan kategori dan status, serta membentuk ringkasan data.

Function	Tanggung Jawab	Input	Nilai Kembalian atau Keluaran
<code>validasiNilai</code>	Memeriksa apakah nilai valid	nilai	true/false
<code>tentukanKategori</code>	Menentukan kategori berdasarkan rentang nilai	nilai	A, B, C, D, null
<code>tentukanStatus</code>	Menentukan status kelulusan	nilai	Lulus, Tidak lulus, Data tidak valid
<code>buatRingkasan</code>	Menggabungkan nama, nilai, kategori, dan status	nama, nilai	Tabel ringkasan

Kasus	Input Nilai	Kategori Diharapkan	Status Diharapkan	Hasil Aktual	Status
1	0	D	Tidak lulus	D / Tidak lulus	☐
2	59	D	Tidak lulus	D / Tidak lulus	☐
3	60	C	Lulus	C / Lulus	☐
4	69	C	Lulus	C / Lulus	☐

Kasus	Input Nilai	Kategori Diharapkan	Status Diharapkan	Hasil Aktual	Status
5	70	B	Lulus	B / Lulus	☐
6	84	B	Lulus	B / Lulus	☐
7	85	A	Lulus	A / Lulus	☐
8	101	null	Data tidak valid	null / Data tidak valid	☐
9	"80"	null	Data tidak valid	null / Data tidak valid	☐

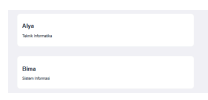
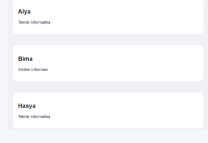
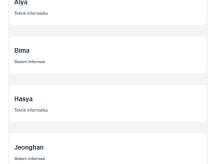
Mengapa 59, 60, 69, 70, 84, dan 85 merupakan boundary value penting?

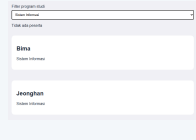
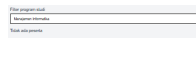
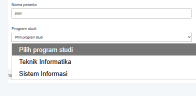
Karena nilai tersebut merupakan nilai yang berada di sekitar batas perubahan kategori, yaitu 59–60 untuk D ke C, 69–70 untuk C ke B, dan 84–85 untuk B ke A. Nilai-nilai ini penting untuk memastikan kondisi pada setiap batas kategori sudah berjalan dengan benar.

Tuliskan perubahan nilai batas yang diuji, prediksi dampaknya, dan hasil pengujian yang diperoleh.

Batas kategori A diubah dari nilai ≥ 85 menjadi nilai ≥ 84 . Diprediksi nilai 84 yang sebelumnya masuk kategori B akan berubah menjadi kategori A. Setelah diuji, nilai 84 memang berubah menjadi A.

D2. Task 2 — DOM dan Form Controller

Skenario	Tindakan	Hasil yang Diharapkan	Hasil Aktual	Bukti
Awal	Muat halaman	Dua kartu peserta tampil	Dua kartu Alya dan Bima tampil	
Kosong	Submit tanpa isi	Dua pesan error; data tidak bertambah	Dua pesan error tampil dan data tidak bertambah	
Nama pendek	Nama 2 karakter	Error hanya pada nama	Error hanya pada nama tampil	
Valid	Isi semua field	Satu kartu baru ditampilkan	Satu kartu peserta baru ditampilkan	
Submit kedua	Tambah peserta lain	Data lama tidak terduplikasi	Data lama tetap ada dan tidak terduplikasi	

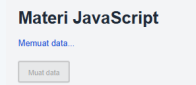
Skenario	Tindakan	Hasil yang Diharapkan	Hasil Aktual	Bukti
Filter ada	Pilih satu prodi	Hanya program studi tersebut yang ditampilkan	Hanya peserta dari prodi yang dipilih tampil	
Filter kosong	Pilih Manajemen Informatika	Pesan "Tidak ada peserta" ditampilkan	Pesan "Tidak ada peserta" tampil	
Keyboard	Tab lalu Enter	Seluruh kontrol dapat digunakan melalui keyboard	Form dan filter dapat digunakan melalui keyboard	

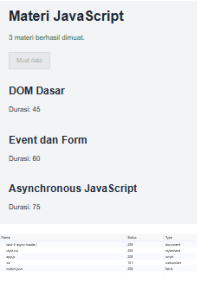
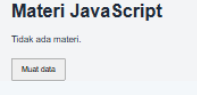
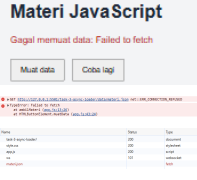
Keputusan Teknis	Penjelasan / Bukti
Cara menggunakan createElement dan textContent	createElement digunakan untuk membuat elemen article, h2, dan p, lalu textContent digunakan untuk mengisi teks tanpa innerHTML.
Cara replaceChildren mencegah duplikasi	replaceChildren() mengosongkan isi daftar sebelum kartu peserta baru ditambahkan, sehingga kartu tidak menumpuk atau terduplikasi.
Cara validasi dipisahkan dari render DOM	validasiPeserta() hanya memeriksa data dan mengembalikan hasil validasi, sedangkan renderPeserta() menangani tampilan peserta.
Cara aria-invalid diperbarui	Nilai aria-invalid diatur menjadi true ketika field tidak valid dan false setelah input diperbaiki.
Cara filter menjaga array peserta asli	Filter menggunakan peserta.filter() untuk membuat hasil baru yang ditampilkan, sehingga array peserta asli tidak diubah.

Kesalahan program yang paling penting yang ditemukan pada Task 2 serta cara memperbaikinya:

Kesalahan yang ditemukan adalah daftar peserta dapat berpotensi terduplikasi jika isi daftar tidak dikosongkan sebelum proses render ulang. Kesalahan ini diperbaiki dengan menggunakan replaceChildren() pada renderPeserta() sebelum menambahkan kartu peserta baru.

D3. Task 3 — Asynchronous JavaScript Data Loader

Skenario	Cara Membuat	State dan Hasil yang Diharapkan	Hasil Aktual	Bukti
Idle	Muat halaman	Belum ada permintaan; tombol aktif	Halaman dimuat dengan state awal dan tombol Muat data aktif.	
Loading	Klik Muat data	Pesan loading; tombol dinonaktifkan	Muncul pesan "Memuat data..." dan tombol menjadi tidak aktif selama proses fetch.	

Skenario	Cara Membuat	State dan Hasil yang Diharapkan	Hasil Aktual	Bukti
Success	JSON valid	Tiga kartu tampil; status 200	Tiga kartu materi tampil dan Network menunjukkan status 304	
Empty	JSON = []	Pesan data kosong; bukan error	Muncul pesan "Tidak ada materi." dan tidak muncul kartu.	
Path salah	Ubah URL fetch	State error dan status 404 terlihat pada Network	State berubah menjadi error, pesan error tampil, dan Network menunjukkan status 404.	
JSON rusak	Rusak sintaks JSON	Parsing gagal dan state berubah menjadi error	Data gagal diproses, state berubah menjadi error, dan detail parsing muncul di Console.	
Server mati	Hentikan local server	Fetch ditolak; antarmuka menampilkan error	Fetch gagal dan state berubah menjadi error, sementara detail error tercatat di Console.	
Klik ganda	Klik tombol dua kali dengan cepat	Satu proses; daftar tidak ganda	Tombol langsung dinonaktifkan sehingga proses tidak dijalankan dua kali dan daftar tidak tergandakan.	

Pemeriksaan melalui Network dan Console	Temuan
URL permintaan dan status HTTP	data/materi.json dengan status 200 saat data berhasil dimuat.
Respons JSON	Berisi 3 data materi dalam bentuk array.
Pesan error akibat path atau status HTTP	Saat path materi.json dibuat salah, Network menunjukkan 404 Not Found dan Console menampilkan error HTTP 404.

Pemeriksaan melalui Network dan Console	Temuan
Pesan error parsing	Saat isi JSON dibuat tidak valid, Network tetap menerima file, tetapi Console menampilkan error parsing JSON dari response.json().
Jumlah permintaan ketika tombol diklik dua kali dengan cepat	Hanya satu proses/request yang dijalankan karena tombol dinonaktifkan saat loading.

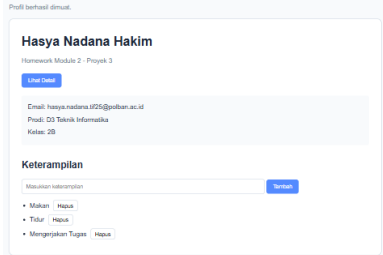
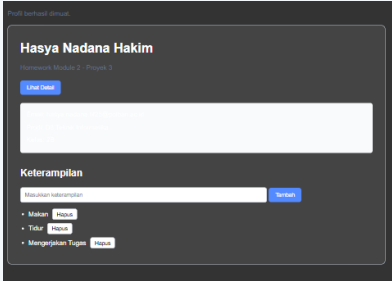
Jelaskan urutan state yang Anda amati sejak tombol diklik hingga seluruh proses selesai:

Saat tombol Muat data diklik, state berubah dari idle menjadi loading dan tombol dinonaktifkan. Setelah data berhasil diambil, state berubah menjadi success jika terdapat data atau empty jika data kosong. Jika terjadi kesalahan, state berubah menjadi error. Setelah proses selesai, tombol diaktifkan kembali melalui finally

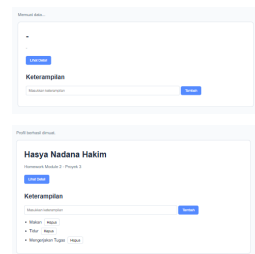
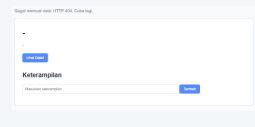
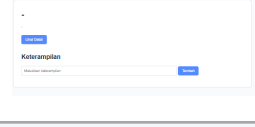
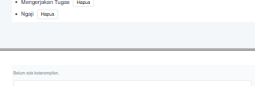
Mengapa Coba Lagi sebaiknya memanggil function pemuatan yang sama?

Karena Coba Lagi perlu menjalankan proses pemuatan data yang sama seperti tombol Muat data. Dengan memanggil muatData(), proses fetch, pengaturan state, penanganan error, dan pengaktifan kembali tombol tetap dilakukan dengan cara yang sama tanpa menulis kode yang sama dua kali

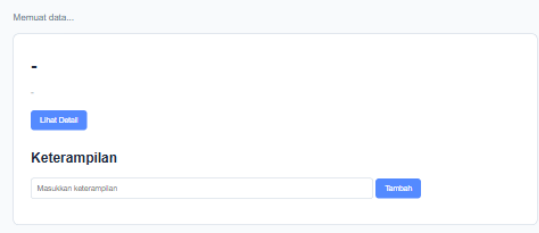
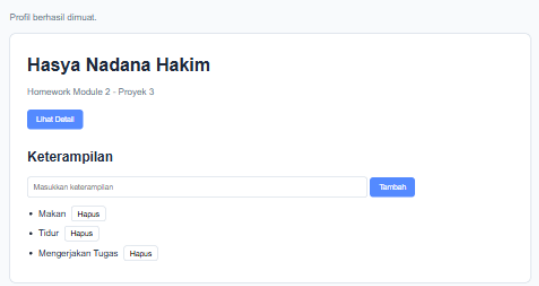
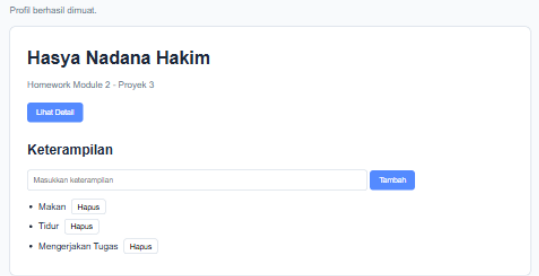
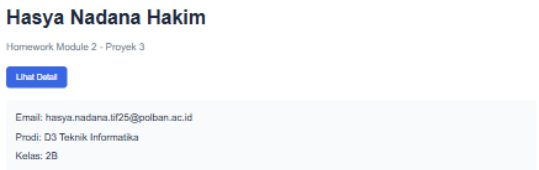
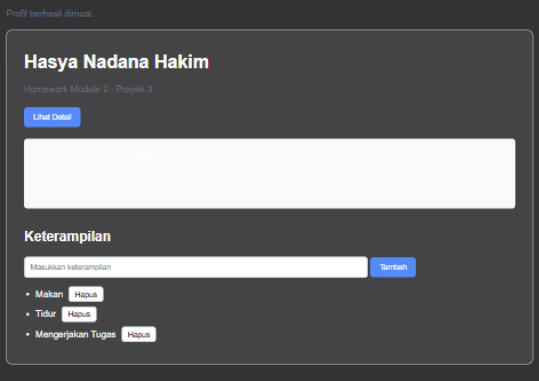
D4. Homework — Interactive Profile Card

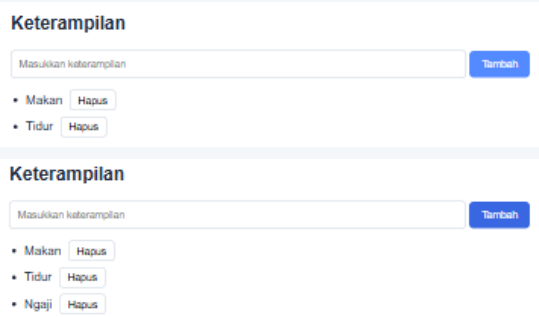
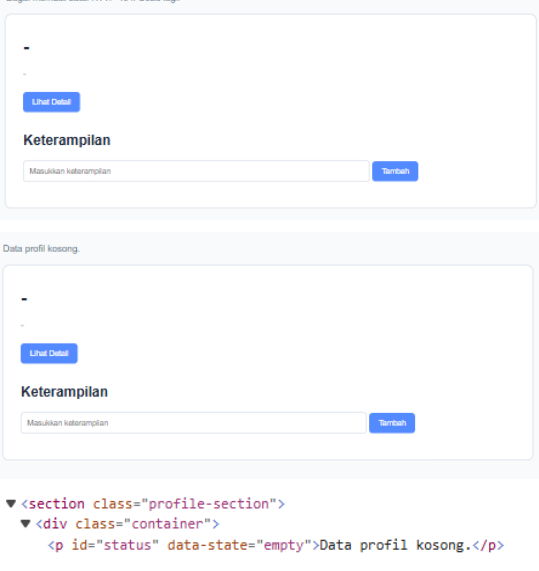
Fitur atau Kondisi	Status	Bukti dan Catatan
Profil dan keterampilan awal dimuat dari data/profile.json	☐ Selesai	 Profil dan keterampilan tampil saat halaman dibuka.
State loading, empty, error, dan Coba Lagi tersedia	☐ Selesai	Loading saat data dimuat, empty saat data kosong, error saat HTTP error, dan Coba Lagi saat fetch gagal.
Detail toggle dengan classList dan aria-expanded	☐ Selesai	Detail terbuka dan tertutup saat tombol diklik.
Tema dapat diganti	☐ Selesai	 Tema berubah saat tombol Ganti Tema diklik.

Fitur atau Kondisi	Status	Bukti dan Catatan
Form penambahan keterampilan memvalidasi input kosong	☐ Selesai	Pesan error muncul jika input kosong.
Setiap keterampilan dapat dihapus	☐ Selesai	Tombol Hapus menghilangkan keterampilan yang dipilih.
Klik berulang tidak menggandakan data	☐ Selesai	Data tidak terduplikasi saat proses render dilakukan kembali.
README menjelaskan cara menjalankan local server	☐ Selesai	README berisi langkah menjalankan project melalui local server.

Skenario	Ekspektasi	Hasil Aktual	Bukti
Halaman dibuka	Loading lalu profil tampil tanpa error	Loading muncul lalu profil tampil	
Path JSON salah	Pesan error dan Coba Lagi tampil	State error dengan pesan HTTP 404 tampil	
JSON kosong	State empty ditampilkan	Pesan 'Data profil kosong' tampil	
Detail diklik dua kali	Terbuka lalu tertutup	Detail terbuka pada klik pertama dan tertutup pada klik kedua	
Input kosong	Data tidak ditambah; pesan muncul	Data tidak bertambah dan pesan error muncul	
Keterampilan yang valid	Item baru muncul satu kali	Keterampilan baru muncul di daftar	
Semua keterampilan dihapus	Pesan untuk kondisi kosong ditampilkan	Pesan 'Belum ada keterampilan.' tampil	

Daftar minimal tujuh tangkapan layar pengujian yang disertakan:

No.	Tangkapan layar	Skenario yang Dibuktikan
1	 	State loading dan profil berhasil dimuat
2		Profil dan keterampilan awal dari profile.json
3		classList.toggle() dan aria-expanded
4		Fitur Ganti Tema
5		Validasi input kosong

No.	Tangkapan layar	Skenario yang Dibuktikan
6		Tambah keterampilan dan tombol Hapus
7	 <pre> ▼ <section class="profile-section"> ▼ <div class="container"> <p id="status" data-state="empty">Data profil kosong.</p> </pre>	Error + Coba Lagi atau empty saat data kosong

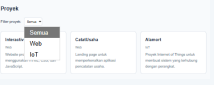
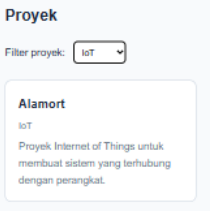
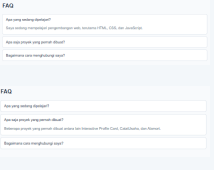
E. Mini Project — Landing Page Interaktif

Batas Teknis Mini project menggunakan HTML, CSS, dan vanilla JavaScript. Belum menggunakan fetch, Promise, atau async/await; asynchronous dinilai pada Task 3 dan homework.

E1. Ringkasan Produk

Aspek	Isian Mahasiswa
Nama atau topik landing page	Portofolio
Target pengguna	Dosen, recruiter, dan orang yang ingin melihat profil serta hasil karya mahasiswa
Masalah atau kebutuhan pengguna	Membutuhkan media untuk menampilkan profil, keterampilan, proyek, dan informasi kontak secara ringkas dan interaktif
URL repository	https://github.com/shiningsstars/Proyek-3-Module-2.git
URL deployment (jika ada)	

E2. Rencana Fitur dan Acceptance Criteria

Fitur Minimal	Perilaku atau State yang Direncanakan	Acceptance Criteria Mahasiswa	Bukti
Navigasi mobile	Menu navigasi dapat dibuka dan ditutup melalui button pada layar kecil.	Button dapat membuka dan menutup menu navigasi.	
Daftar fitur atau layanan dinamis	Daftar proyek ditampilkan berdasarkan array of objects.	Semua proyek pada array ditampilkan sebagai card dan tidak ada data yang terduplikasi.	
Filter daftar	Proyek dapat difilter berdasarkan kategori.	Hanya proyek yang sesuai dengan kategori yang dipilih ditampilkan.	
FAQ accordion	Pertanyaan dapat dibuka dan ditutup untuk melihat jawaban	Jawaban FAQ terbuka saat dipilih dan aria-expanded berubah sesuai kondisi.	
Form kontak beserta validasi	Pengguna dapat mengisi form untuk mengirim pesan.	Input kosong atau tidak valid ditolak dan pesan keberhasilan muncul jika input valid.	
Fitur kembali ke atas atau interaksi lain yang relevan	Tombol kembali ke atas digunakan setelah pengguna melakukan scroll.	Tombol dapat membawa pengguna kembali ke bagian atas halaman.	
Tema atau perubahan visual berbasis class	Pengguna dapat mengubah tampilan halaman menjadi tema gelap.	Class tema ditambahkan/dihapus dan tampilan halaman berubah.	

E3. Pemetaan Event, Fungsi Penanganan, dan Perubahan DOM

Fitur	Event	Fungsi Penanganan	Elemen DOM atau Class yang Berubah	State
Navigasi mobile	click	toggleMenu()	Class menu, aria-expanded	Terbuka / tertutup
Filter	change	filterProyek()	Daftar proyek	Semua / terfilter / kosong
FAQ	click	toggleFAQ()	Class jawaban, aria-expanded	Terbuka / tertutup

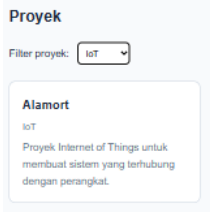
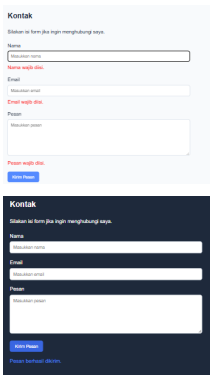
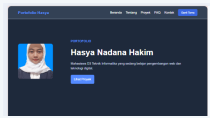
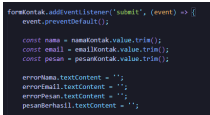
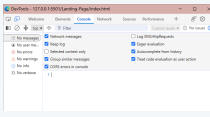
Fitur	Event	Fungsi Penanganan	Elemen DOM atau Class yang Berubah	State
Form	submit	prosesForm()	Pesan error / keberhasilan	Valid / tidak valid / berhasil
Tema atau state	click	toggleTema()	Class pada body	Normal / gelap

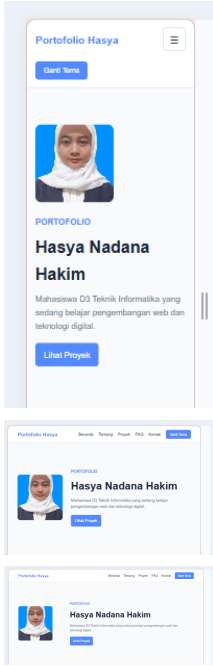
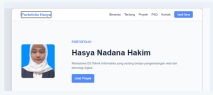
E4. Riwayat Tahapan

Tahap	Tanggal	Keluaran atau Perubahan	Bukti atau Commit
Audit Modul 1	170926	Menentukan konsep baru berupa landing page portofolio dan struktur halaman yang akan digunakan.	
Acceptance criteria	170926	Menentukan fitur-fitur interaktif beserta kriteria keberhasilannya.	
Selector dan data awal	170926	Menentukan selector HTML dan data awal proyek yang akan digunakan oleh JavaScript.	
Fitur 1 sampai 3	170926	Membuat menu mobile buka/tutup, render daftar proyek dari array of objects, dan filter proyek berdasarkan kategori.	
Fitur 4 sampai 7	180926	Membuat FAQ accordion, validasi form kontak, tombol kembali ke atas, dan fitur perubahan tema.	
Pengujian keyboard dan tampilan responsif	180926	Menguji fungsi navigasi, tombol, form, filter, serta tampilan halaman pada ukuran layar berbeda.	
Perbaikan dan refaktorisasi	180926	Memperbaiki tampilan dan fungsi yang belum sesuai serta merapikan kode JavaScript dan CSS.	
Code selesai	180926	Menyelesaikan seluruh fitur interaktif dan memastikan halaman dapat digunakan tanpa error.	

E5. Acceptance Test Mini Project

ID	Kriteria	Status	Bukti dan Catatan
MP-01	Navigasi mobile: button dapat membuka dan menutup menu serta memperbarui aria-expanded secara sesuai.	☐ Lulus	
MP-02	Daftar dinamis: jumlah kartu mengikuti data dan tidak duplikat	☐ Lulus	

ID	Kriteria	Status	Bukti dan Catatan
MP-03	Filter: subset tepat dan kondisi kosong ditangani	☐ Lulus	
MP-04	FAQ: jawaban terkait terbuka, lainnya tertutup, keyboard bekerja	☐ Lulus	
MP-05	Form: input yang tidak valid ditolak, sedangkan input yang valid menampilkan ringkasan atau pesan keberhasilan.	☐ Lulus	
MP-06	Tema atau state: class berubah dan kontras teks tetap terbaca.	☐ Lulus	
MP-07	Tidak ada inline onclick dan tidak memakai innerHTML untuk input pengguna	☐ Lulus	
MP-08	Tidak ada error Console pada alur normal	☐ Lulus	

ID	Kriteria	Status	Bukti dan Catatan
MP-09	Viewport 320, 768, dan 1024 px tidak merusak tampilan Modul 1	☐ Lulus	
MP-10	Fitur utama dapat digunakan dengan mouse dan keyboard	☐ Lulus	
MP-11	Folder proyek, worksheet, README, dan bukti uji lengkap	☐ Lulus	
MP-12	Saya dapat menjelaskan alur event sampai perubahan DOM	☐ Lulus	

F. Kuesioner Self-Assessment

Skala: 1 belum mampu; 2 mampu dengan banyak bantuan; 3 mampu dengan panduan; 4 mandiri; 5 mampu menjelaskan kepada orang lain.

No.	Pernyataan	1	2	3	4	5
1	Saya dapat menggunakan tipe data, kondisi, dan function.	☐	☐	☐	☒	☐
2	Saya dapat mengolah array of objects.	☐	☐	☒	☐	☐
3	Saya dapat memilih dan mengubah elemen DOM.	☐	☐	☒	☐	☐
4	Saya dapat memasang event listener dan menjelaskan alurnya.	☐	☐	☒	☐	☐
5	Saya dapat memvalidasi form dan menampilkan pesan.	☐	☐	☒	☐	☐
6	Saya dapat menjelaskan sinkron dan asinkron.	☐	☒	☐	☐	☐
7	Saya dapat menggunakan Promise dan async/await.	☐	☒	☐	☐	☐
8	Saya dapat memakai fetch dan memeriksa response.ok.	☐	☒	☐	☐	☐
9	Saya dapat menangani loading, success, empty, dan error.	☐	☐	☒	☐	☐

No.	Pernyataan	1	2	3	4	5
10	Saya dapat membaca serta memperbaiki error JavaScript.	☐	☒	☐	☐	☐
11	Saya dapat menjelaskan seluruh kode yang dikumpulkan.	☐	☐	☒	☐	☐

Bagian paling menantang pada Modul 2:

Bagian yang paling menantang adalah memahami cara JavaScript berinteraksi dengan DOM, terutama penggunaan event listener, classList, dan perubahan isi elemen. Saya juga cukup perlu memahami proses fetch() dan state seperti loading, success, empty, dan error.

Konsep yang sekarang paling siap saya jelaskan kepada orang lain:

Saya paling siap menjelaskan konsep fetch() dan cara kerjanya dalam mengambil data dari file JSON. Mulai dari fetch() untuk meminta data, response.ok untuk mengecek keberhasilan permintaan, response.json() untuk membaca data JSON, sampai try dan catch untuk menangani error.

G. Peer Assessment

Nama Penilai	Nama Pemilik Mini Project
R Neysa Velda	Hasya Nadana Hakim

No.	Kriteria	Skor 1 sampai 5	Catatan Kelebihan dan Saran Perbaikan
1	Kesesuaian fitur: seluruh fitur minimal tersedia	4	Fitur minimal sudah tersedia dan dapat digunakan.
2	Daftar dinamis: data, render, filter, kondisi kosong bekerja	4	Daftar proyek berhasil ditampilkan dari array dan filter berjalan sesuai kategori.
3	Interaksi: state konsisten dan dapat diulang	3	Interaksi utama sudah berjalan, tetapi beberapa bagian masih dapat dirapikan agar lebih konsisten dan sesuai ketentuan.
4	Validasi: input salah ditangani dan pesan membantu	4	Validasi form sudah berjalan dan pesan error mudah dipahami.
5	Keyboard: navigasi, FAQ, dan form dapat digunakan tanpa mouse	4	Navigasi, FAQ, dan form dapat digunakan dengan keyboard.

No.	Kriteria	Skor 1 sampai 5	Catatan Kelebihan dan Saran Perbaikan
6	Stabilitas: tidak ada error atau duplikasi elemen	4	Fitur berjalan tanpa error dan elemen tidak mengalami duplikasi.

Satu umpan balik dari rekan yang akan saya terapkan:

Navigasi dibuat menjadi hamburger dan respons FAQ disesuaikan

H. Log Bantuan dan Penggunaan AI

Prinsip AI boleh membantu menjelaskan konsep atau memberi hint setelah ada percobaan. Jawaban AI harus diverifikasi melalui dokumentasi, DevTools, Network, atau eksperimen mandiri.

Tanggal atau Fase	Pertanyaan atau Sumber	Intisari Bantuan	Hasil Verifikasi	Penyesuaian atau Perubahan yang Saya Lakukan
170926 / Event dan Form	Meminta bantuan memahami addEventListener() dan event.preventDefault().	Menjelaskan cara menangani event dan mencegah perilaku bawaan form.	Form diuji dengan dan tanpa preventDefault() untuk melihat perbedaannya.	Mengembalikan preventDefault() setelah melihat halaman melakukan reload.
170926 / Validasi	Meminta bantuan menyusun fungsi validasi nama dan email.	Menjelaskan penggunaan trim(), kondisi if, pesan error, dan aria-invalid.	Input kosong, nama pendek, email tanpa @, dan data valid diuji langsung.	Menyesuaikan pesan error agar lebih mudah dipahami.
180926 / FAQ Accordion	Meminta bantuan memahami cara membuka dan menutup jawaban FAQ menggunakan classList.	Menjelaskan penggunaan classList.add(), classList.remove(), dan aria-expanded.	FAQ diuji dengan klik dan keyboard untuk melihat perubahan state.	Membuat hanya satu jawaban yang terbuka pada satu waktu.
180926 / Async Data Loader	Meminta bantuan memahami Promise, async/await, fetch(), dan state.	Menjelaskan alur pemuatan data, delay, error, serta state loading, empty, dan error.	Diuji melalui browser, Console, dan Network dengan data normal, JSON kosong, path salah, dan simulasi error.	Menyesuaikan kode dengan kebutuhan tugas dan menggunakan replaceChildren() untuk mencegah hasil render terduplikasi.

Jika tidak menggunakan AI, jelaskan strategi debugging dan dokumentasi yang digunakan:

H1. Bukti Pemahaman

Pilih satu saran yang Anda terima. Jelaskan kembali konsepnya dengan bahasa sendiri:

Saya mendapat saran untuk menggunakan `event.preventDefault()` pada form agar perilaku bawaan browser tidak langsung menjalankan reload halaman. Dengan begitu, data dari form dapat diperiksa dan diproses terlebih dahulu menggunakan JavaScript.

Eksperimen yang membuktikan saran tersebut benar atau tidak:

Saya mencoba menjalankan form dengan `event.preventDefault()` dan kemudian menonaktifkan kode tersebut. Saat `preventDefault()` digunakan, halaman tidak melakukan reload dan proses validasi dapat dijalankan oleh JavaScript. Saat kode tersebut dihilangkan, halaman melakukan reload setelah form dikirim. Dari percobaan tersebut saya memahami fungsi `preventDefault()` dalam menangani submit form.

H2. Batas yang Saya Patuhi

- ☐ Saya menulis prediksi dan percobaan awal sendiri.
- ☐ AI tidak menghasilkan seluruh Task 1, Task 2, Task 3, homework, atau mini project.
- ☐ Pada debugging, saya membaca error, baris, selector, atau nilai aktual sebelum meminta hint.
- ☐ Prompt dan dampaknya dicatat ketika AI digunakan.
- ☐ Saya dapat menjelaskan dan mengubah kode tanpa AI saat coding.

I. Lesson Learnt

Konsep JavaScript: Konsep JavaScript apa yang sekarang dapat Anda jelaskan dengan contoh?

Saya sekarang dapat menjelaskan penggunaan `fetch()` untuk mengambil data dari file JSON, termasuk mengecek `response.ok` dan membaca data dengan `response.json()`.

Sinkron dan Asinkron: Apa perbedaan terbesar yang Anda rasakan antara kode sinkron dan asinkron?

Perbedaan yang paling terasa adalah kode asinkron tidak harus menunggu proses selesai untuk melanjutkan bagian kode lainnya. Contohnya `fetch()` membutuhkan waktu untuk mengambil data, sehingga digunakan `async` dan `await` agar proses tersebut dapat ditunggu tanpa menghentikan seluruh halaman.

Kesalahan yang Memberikan Pembelajaran: Error fetch atau JSON apa yang paling membantu pemahaman?

Error HTTP 404 saat path JSON salah paling membantu karena saya bisa melihat hubungan antara kesalahan URL dengan hasil `fetch()`. Saya juga belajar bahwa JSON yang tidak valid dapat menyebabkan error saat diproses dengan `response.json()`.

Keputusan Teknis: Keputusan teknis apa yang Anda ambil dan apa alasannya?

Saya memisahkan fungsi berdasarkan tugasnya, misalnya fungsi untuk mengambil data, merender data, dan menangani event. Saya juga menggunakan `createElement()` dan `textContent` saat membuat elemen dari data agar struktur dan isi halaman lebih mudah dikontrol.

Kemandirian: Bagian mana yang diselesaikan tanpa bantuan?

Saya dapat melakukan pengujian pada kode, mencoba kondisi normal maupun error, serta mengecek hasilnya melalui browser dan Console. Saya juga dapat memperbaiki beberapa kesalahan berdasarkan hasil pengujian tersebut.

Umpan Balik Rekan: Umpan balik peer apa yang anda terapkan?

Navigasi dibuat menjadi hamburger dan respons FAQ disesuaikan

Target Berikutnya: Kompetensi apa yang menjadi target berikutnya?

Target berikutnya adalah lebih terbiasa membuat fitur interaktif dari awal sampai selesai, mulai dari menentukan struktur HTML, membuat logika JavaScript, menghubungkan data, sampai melakukan pengujian dan debugging.

J. Checklist Akhir Modul 2

J1. Kualitas Kode dan Interaksi

- ☐ File JavaScript dipanggil dengan defer dan tidak menghasilkan error saat halaman dibuka.
- ☐ Nama variabel dan function menjelaskan isi atau tindakan.
- ☐ Input dikonversi dan divalidasi sebelum digunakan.
- ☐ Function logika dapat diuji terpisah dari DOM.
- ☐ Selector sesuai dengan HTML dan tidak ada null yang tidak dijelaskan.
- ☐ Interaksi menggunakan addEventListener, bukan atribut event inline.
- ☐ Perubahan visual memakai class CSS.
- ☐ Button dan form dapat digunakan dengan keyboard.
- ☐ Pesan error spesifik dan tidak hanya mengandalkan warna.
- ☐ Proses asynchronous memiliki state idle, loading, success, empty, dan error.
- ☐ fetch memeriksa response.ok dan menangani kegagalan HTTP atau parsing.
- ☐ Tidak ada pesan error yang belum dijelaskan pada Console.

J2. Kelengkapan Pengumpulan

Komponen	Status	Lokasi atau Nama File
Worksheet PDF	☐	251511040_Hasya Nadana Hakim_Worksheet_Modul2
Task 1 beserta bukti pengujian	☐	task-1-logic-builder
Task 2 beserta tangkapan layar skenario pengujian	☐	task-2-dom-form
Task 3 beserta bukti dari Console dan Network	☐	task-3-async-loader
Homework beserta minimal tujuh tangkapan layar pengujian	☐	Homework 2
Mini project beserta README	☐	Landing-Page
Log bantuan dan penggunaan AI	☐	251511040_Hasya Nadana Hakim_Worksheet_Modul2
Repository atau deployment dapat diakses	☐	https://github.com/shiningsstars/Proyek-3-Module-2.git

Catatan akhir sebelum mengumpulkan:

--